

From alert to answer

Your alert says revenue moved.

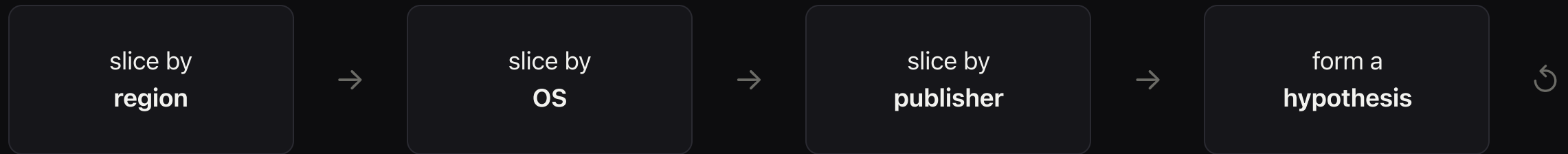
This says **why** — and what it **ruled out**.

An automated root-cause analyst, built on ClickHouse.
github.com/shravankgl/inmobi-rca-clickathon-2026 · MIT

THE PROBLEM

Every data-driven team watches a handful of numbers that matter. The question is *why*.

TODAY, THAT ANSWER COSTS AN ANALYST HOURS



And the decision waiting on it — pause a campaign, escalate to an SDK team, or do nothing because it is seasonal — **should take minutes.**

Point it at a window. Thirty seconds later, a diagnosis — and every dimension it checked, with the numbers.

Incidents

6 found, most severe first

LOCALIZED

sev 256.2

2026-07-06 00:00:00 → 2026-07-10 21:00:00 (118h)

Revenue -2.1%, driven by ecpm in ad_format=video (-18.0% vs -9.1% across the platform)

LOCALIZED

sev 32.3

2026-06-23 00:00:00 → 2026-06-25 23:00:00 (72h)

Revenue -0.2%, driven by fill_rate in os_version=Android 15 (-44.9% vs -4.4% across the platform)

GLOBAL

sev 11.3

2026-06-21 00:00:00 → 2026-06-21 23:00:00 (24h)

Revenue -44.8%, driven by requests falling uniformly across every dimension — no single segment responsible

UNATTRIBUTED

sev 8.8

2026-06-28 04:00:00 → 2026-06-30 23:00:00 (68h)

Revenue +6.0%, driven by requests; no dimension met the attribution bar

27 compound findings in this window — see "Compound segments" below

UNATTRIBUTED

sev 8.6

2026-06-18 23:00:00 → 2026-06-19 23:00:00 (25h)

Revenue +0.6%, driven by requests; no dimension met the attribution bar

UNATTRIBUTED

sev 1.1

2026-06-17 10:00:00 → 2026-06-17 20:00:00

52,931 ms query time1,218,035,028 rows read131 queries101.3s runtime

Metric tree

Funnel

Diagnosis

Checked & ruled out

Responsible segments

ad_format = video

29% of the whole incident · 100% of the unexplained movement

video moved -18.0% against -9.4% globally, carrying 100% of the unexplained movement and 29% of the whole incident

os_version = Android 12

also flagged

20% of the whole incident · 73% of the unexplained movement

Android 12 moved -17.2% against -9.3% globally, carrying 73% of the unexplained movement and 20% of the whole incident

device_model = iPhone 14

also flagged

18% of the whole incident · 66% of the unexplained movement

iPhone 14 moved -22.3% against -9.2% globally, carrying 66% of the unexplained movement and 18% of the whole incident

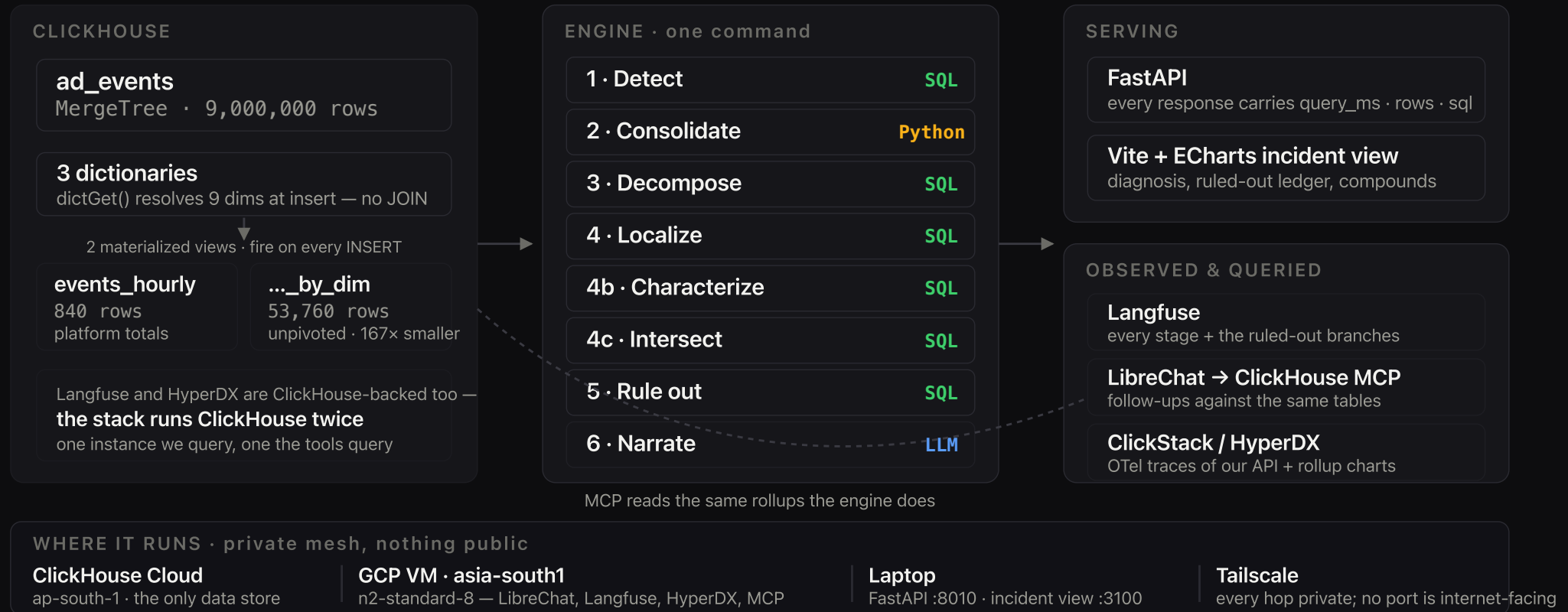
Ranked by share of the incident. A dimension correlated with the primary will move with it — check whether the secondary is an independent cause or a reflection of the first before acting on it.

Checked and ruled out

Dimension	Verdict	Top value	Reason
region	INCONCLUSIVE	NAM	NAM leads this dimension, but its excess is 0% of the unexplained movement, under the 50% concentration bar — the movement is spread across values, not concentrated in one
country	INCONCLUSIVE	US	US leads this dimension, but its excess is 0% of the unexplained movement, under the 50% concentration bar — the movement is spread across values, not concentrated in one
publisher_tier	UNIFORM	tier_2	all 3 values moved together (every one within 0.5% of the global -9.1%); the largest outlier explains only 1.1% of the incident, so this dimension does not explain it
category	UNIFORM	gaming	all 7 values moved together (every one within 3.7% of the global -9.3%); the largest outlier explains only 0.8% of the incident, so this dimension does not explain it
campaign_type	UNIFORM	CPM	all 4 values moved together (every one within 9.1% of the global -9.1%); the largest outlier explains only 0.6% of the incident, so this dimension does not explain it
vertical	UNIFORM	travel	all 8 values moved together (every one within 9.1% of the global -9.1%); the largest outlier explains only 0.3% of the incident, so this dimension does not explain it

3

Everything is ClickHouse. Seven stages are SQL. One is the LLM.



Delete stage 6 and the diagnosis is unchanged. The LLM receives computed numbers, never an event row.

We built the obvious rollup first, and measured it.

FULLY CROSSED · HOUR × 9 DIMS

~~7,247,816~~

rows from 7.2M events.

Nearly every event its own combination.

It compressed nothing.

UNPIVOTED · (HOUR, DIM_NAME, DIM_VALUE)

53,760

Only 62 distinct dimension values exist in total.

167× smaller — and exactly the shape contribution ranking reads.

SummingMergeTree, not Aggregating — every measure is additive, so `-State/-Merge` would buy nothing. **Dictionaries**, not JOINS — nine dimensions resolved by `dictGet()` at insert time. **No statistic is materialized** — the baseline window is a parameter; freezing it at write time returns the wrong answer.

SPEED

Measured, best of three, from the query log

QUERY	ROWS SCANNED	TIME	PATH
Detect: hourly revenue, 5 weeks	840	37.4 ms	rollup
<i>same, from raw</i>	<i>9,000,000</i>	<i>114.0 ms</i>	<i>raw</i>
Rank: fill rate by os_version, 3 days	8,192	38.5 ms	rollup
<i>same, from raw</i>	<i>851,968</i>	<i>57.7 ms</i>	<i>raw</i>

10,714x

fewer rows scanned. The wall-clock gap is only ~3x — at 9M rows ClickHouse is fast either way. Rows are what makes **131 queries per run** viable.

131

queries · **1,218,035,028 rows** · **52.9 s** of ClickHouse time across both datasets, from **168 lines** of hand-written SQL.

One ARRAY JOIN fans every event into one row per dimension

```
CREATE MATERIALIZED VIEW inmobi.mv_events_hourly_by_dim
TO inmobi.events_hourly_by_dim AS
SELECT toStartOfHour(event_time) AS hour, d.1 AS dim_name, d.2 AS dim_value,
       count() AS requests, sum(is_filled) AS fills,
       sum(is_impression) AS impressions, sum(revenue) AS revenue
FROM inmobi.ad_events
ARRAY JOIN [
  ('os_version', dictGetString('inmobi.dict_geo_device','os_version', geo_device_id)),
  ('region',      dictGetString('inmobi.dict_geo_device','region',      geo_device_id)),
  ('category',    dictGetString('inmobi.dict_apps','category',          app_id)),
  ...9 dimensions
] AS d
GROUP BY hour, dim_name, dim_value;
```

The dictionaries resolve every dimension **at insert time**, so the view never JOINS. Ratios are never stored — fill rate, CTR and eCPM are recomputed as sum/sum on read, because averaging per-hour ratios is silently wrong the moment bucket sizes differ.

Three anomalies, three different shapes

WHEN	SHAPE	DIAGNOSIS
06-21 24 h	GLOBAL uniform across all 9 dims	requests -43.5% ; fill, render and eCPM normal. Traffic arrival — no segment responsible .
06-23 → 25 72 h	LOCALIZED os_version = Android 15	fill rate 0.785 → 0.433 ; requests normal. Invisible globally — moved platform fill only ~5%.
06-28 → 30 3 days	COMPOUND iOS 18.1 × APAC	fill rate -50.6% . No single-dimension scan can see it.

Same day. Four ways of looking.

global	−1.0%
APAC alone	−2.3%
iOS 18.1 alone	−12.3%
iOS 18.1 × APAC	−50.6%

Neither parent moved on its own. **Only the cell did.**

This is the problem statement's own worked example — *"a drop in fill rate for Device X in Region North."* We were missing it until a reviewer pointed out our localizer only looked at one dimension at a time.

Saying “no segment is responsible” *is* the finding

DIMENSION	VERDICT	BECAUSE
publisher_tier	UNIFORM	all 3 values within 0.2% of the global -4.4%
ad_format	UNIFORM	all 5 values within 0.4%
category	UNIFORM	all 7 values within 0.5%
campaign_type	NOT APPLICABLE	fill rate is 1.0 by construction — the field only exists once a request has filled

The bug this prevents. Ranking segments by size of drop named tier_1 responsible for a **315-request** deviation out of an incident of **97,027** — 0.3%. Confident, reproducible, wrong.

1.5M fresh events. One command, 37 seconds.

A · JUL 6 — MIX SHIFT

Request **share** moved: APAC + LATAM **+17pp**, EU + NAM **-13pp**. The two cheapest regions displaced the two most expensive.

No region's own eCPM changed. Global eCPM still fell ~8% — dilution, not a price change.

B · JUL 9 — REAL PRICE CHANGE

video eCPM **-30%**, rewarded **+26%**, everything else flat.

Shares did not move. So this one is genuine, not a mix effect.

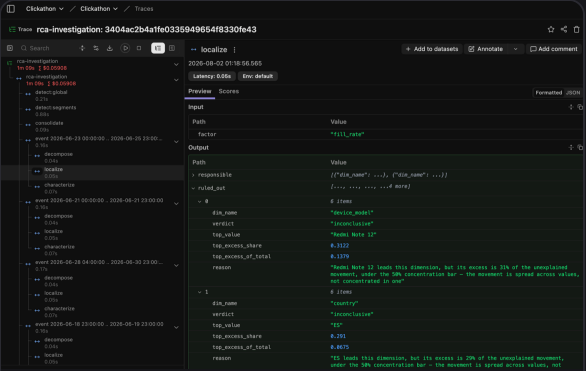
RUN	WHAT OUR SYSTEM SAID	VERDICT
B Jul 9-10	ecpm in ad_format=video -35.0% vs -12.0%	Correct — segment, magnitude, direction
A Jul 6-8	requests in campaign_type=CPC +139.5% vs +8.6%	Right window & factor — named CPC; region=APAC ranked 5th of 7
Full 119h	one event, ecpm in ad_format=video	Partly — merged A and B into one

Why we missed A's cause. Our attribution asks whether a segment's *metric* moved beyond what its size explains. In a mix shift **no segment's metric moves** — only the weights do. Catching it needs a share-of-volume decomposition alongside the metric one. We did not build that.

Four open-source products, four jobs

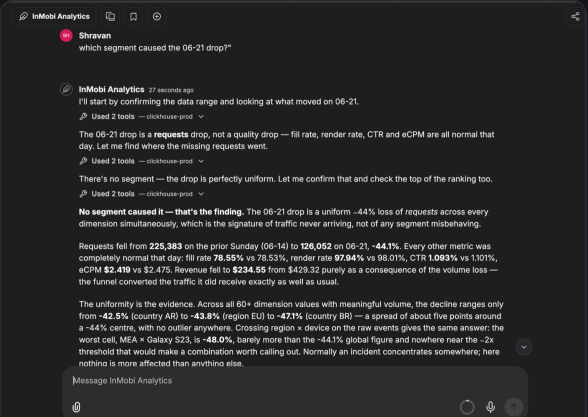
Langfuse

Every stage traced — SQL, rows, timing, verdict — **including the ruled-out branches.**



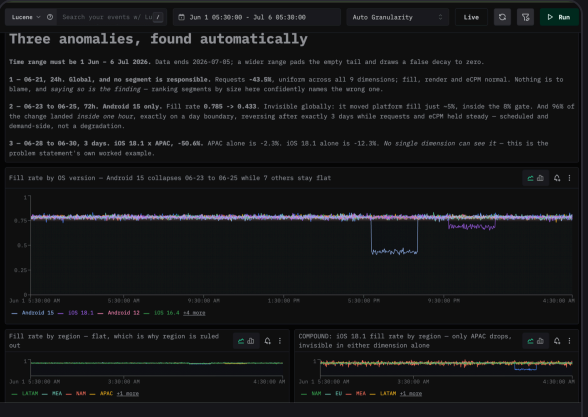
LibreChat + ClickHouse MCP

Same conclusions, different route — and one finding the engine doesn't make.



ClickStack / HyperDX

Charts the rollups directly — the anomaly as a raw shape, not our claim about it.



Three claims

- **It reports what it ruled out, with numbers.** On a uniform drop the correct answer is “no segment is responsible” — and most systems will confidently name one anyway.
- **It finds compound segments** that no single-dimension scan can see.
- **The LLM writes only the sentence.** It receives computed numbers, never an event; every figure in its prose is matched back to the evidence, and the run **exits non-zero** if one cannot be.

AND THE HONEST LIMITS

We see the **shape** of a change, not the system that caused it — we say what the evidence is consistent with and name what a human should check. Compound search is **pairs, not triples**. Thresholds are judgment calls anchored to measurements on one dataset, each listed in `METHOD.md` with how it fails; we have run no sensitivity sweep.

DOES IT WORK IN REAL TIME?

Yes — and it needed no new maths

The materialized views are **insert triggers**: they fire on every INSERT regardless of source — Kafka, ClickPipes, an HTTP POST from an edge collector. No batch job, no cron, no recompute. And because **no statistic is materialized**, there is nothing to drift or rebuild. One pass over a 48-hour window is **~1.2 s**; at a 60-second poll that is under 5% duty cycle.

THE EDGE CASE BATCH TESTING CANNOT SURFACE

Run at **14:30** and the newest hour holds **thirty minutes of traffic**. Against a full-hour baseline that reads as a **~50% collapse** — on every run, forever.

The arithmetic is correct, the baseline is correct, **the comparison is invalid** — exactly the failure this project exists to prevent. It never appears in testing, because the provided dataset ends on a complete hour. It would appear within minutes of going live.

Fixed: an inferred window stops one hour short, because in an append-only stream an hour is only provably complete once a later hour exists. An explicit `--end` is still honoured — the caller stated the window and owns it.

Also handled: alert dedup across passes, so a 3-day incident pages once, not 72 times. **Not built:** incident state that survives restarts, backfill-aware baselines for late-arriving events, per-tenant isolation, alert routing. `PRODUCTION.md` lists each one.

We measured which stage doesn't scale

STAGE	ROWS READ	TIME	READS
Compound scan	1,211,930,496	43.2 s	raw events
Everything else — detect, decompose, localize, rule out	6,104,532	9.7 s	rollups

One stage is **99.5% of the rows**. The rollup holds **53,760 rows at 9M events — and at 900M**, because it grows with distinct values × hours, not events. The fix for the outlier is the trick we already used once: a **pair rollup**, ~33,600 rows.

github.com/shravankgl/inmobi-rca-clickathon-2026

MIT · Shravan & team · InMobi problem statement

Thank you.